GUÍA DE PREPARACIÓN

SEGUNDA AUDITORÍA PRESENCIAL DEL PROYECTO (GUÍA 6, S14)

Hotel Reservas · Sistema Transaccional Móvil (APK)

Universidad Manuela Beltrán · Ingeniería de Sistemas · Sistemas Transaccionales

Docente: Jamilton Fernando Benvides

Integrantes: Gustavo Adolfo Cardona Bayona, Jonathan Soler Guevara, Hugo Andrés Forero Angel

Auditoría presencial: 5 de noviembre de 2026 · Documento de uso interno del equipo (no se entrega)

1. Qué pide la guía y con qué lo cumplimos

La Guía 6 revisa cuatro cosas: el documento maestro en PDF con todos los apartados integrados, el repositorio de GitHub con el código organizado, una demostración funcional en vivo (no capturas) y una explicación técnica por parte de los tres integrantes. Lo que cambia frente a la primera auditoría es el peso de la demostración: el docente quiere ver que un dato ingresado desde la interfaz queda guardado en la base de datos y se puede consultar después. Por eso la APK versión 0.8 incluye el modo conectado, que llama a la API REST y a SQL Server en lugar de simularlos.

1.1 Documento maestro (numeral 2.1 de la guía)

El archivo que se entrega es docs/FaseII_y_III_Documentacion_Proyecto_Hotel_Reservas.pdf del repositorio (110 páginas, un solo documento). La siguiente tabla indica dónde está cada apartado que la guía verifica.

Apartado que verifica la guía	Dónde está en el documento maestro
Planteamiento del problema, justificación y objetivos	Sección 1 (1.3 Problemática o necesidad identificada, 1.4 Justificación), 2 Objetivo central y 3 Objetivos específicos
Requerimientos funcionales y no funcionales	Sección 9 (Tabla 2, RF01 a RF21) y sección 10 (RNF01 a RNF05); matriz de pruebas de los RNF en 16.1 (Tabla 11)
Arquitectura y diseño de la solución	Sección 4 (arquitectura en capas, 4.5 justificación de tecnologías) y 8 (integración de los modelos)
Diagramas y modelos	Secciones 5 (casos de uso), 6 (clases), 7 (MER, modelo relacional, arquitectura de la base de datos)
Marco conceptual, contextual e integrador con el servicio de IA	Secciones 11, 12 y 13 (asistente de consultas) y 15.6 (servicio inteligente TensorFlow)
Estado del arte	15.1
Modelado de una red neuronal simple	15 (propósito en la introducción, Tabla 7 entradas y salida, Figura 18 capas, 15.2 y 15.3 funcionamiento); métricas en 15.5
Presupuesto del proyecto	18.1 (Tablas 15 a 18, en pesos colombianos)
Cronograma de desarrollo	18.2 (Tabla 19 y Figura 26, diagrama de Gantt)
Evidencias y plan de mejora de la primera auditoría	16 (validación técnica) y 17 (ficha del 25 de octubre y compromisos)

1.2 Repositorio GitHub (numeral 2.2)

Enlace: https://github.com/gustavo8015/HotelReservas (77 commits hasta el 29 de octubre, más los de esta entrega). Lo que la guía exige y la carpeta que lo demuestra:

Exigencia	Carpeta o archivo
Código fuente actualizado y organizado	HotelReservas.Api (backend), HotelReservasDemo_AndroidStudio (APK), servicio_inteligente (TensorFlow), database (scripts), docs

Exigencia	Carpeta o archivo
Interfaz de usuario funcional	HotelReservasDemo_AndroidStudio/app/src/main/assets/ prototipo.html (APK v0.8) y MainActivity.kt
Backend con la lógica de negocio	HotelReservas.Api/Controllers (Auth, Disponibilidad, Reservas, Huespedes, Comprobantes, Asistente, Notificaciones, Prediccion) y Servicios/ServicioReservas.cs
Conexión con la base de datos	HotelReservas.Api/Data/HotelReservasContext.cs (Entity Framework Core) y appsettings.json (cadena de conexión)
Scripts de tablas y datos de prueba	database/01 a 07 (01 crea las nueve tablas, 03 carga datos, 02 y 05 procedimientos, 06 contraseñas, 07 servicio inteligente)
Registro y consulta	POST /api/huespedes, POST /api/reservas, GET /api/disponibilidad, GET /api/reservas, GET /api/reservas/{codigo}
Validaciones y manejo de errores	401 credenciales, 400 ErrorNegocio (fechas, capacidad), 409 solapamiento (THROW 50001) y 409 concurrencia optimista (50012), 409 documento duplicado
README actualizado	README.md: descripción, tecnologías, estructura, instrucciones de ejecución de base de datos, API, APK y servicio inteligente
Commits que evidencian la evolución	Historial desde S7 (18 de septiembre) hasta S14, con mensajes por entrega

2. Preparación del entorno (la noche anterior y una hora antes)

La guía recomienda verificar conexión, dependencias y ejecución antes de la revisión. Esta lista se hace completa en el computador que se llevará a la auditoría. Responsable: Jonathan (base de datos y API) con apoyo de Hugo (APK y teléfono).

2.1 Base de datos

1. Abrir SQL Server Management Studio y conectarse a la instancia local (localhost o .\SQLEXPRESS).
2. Si la base ya existe de la primera auditoría, ejecutar solo los scripts que falten. Si se parte de cero, ejecutar en orden database/01_crear_base_de_datos.sql, 02_transaccion_confirmar_reserva.sql, 03_datos_de_prueba.sql, 05_impuestos_comprobante_huesped_frecuente.sql, 06_consulta_asistente.sql y 07_servicio_inteligente.sql.
3. Dejar abierta una ventana de consulta con estas instrucciones, que se ejecutarán durante la demostración: SELECT TOP 5 * FROM RESERVA ORDER BY id_reserva DESC; SELECT TOP 5 * FROM HUESPED ORDER BY id_huesped DESC; SELECT TOP 5 * FROM PAGO ORDER BY id_pago DESC; SELECT TOP 5 * FROM BITACORA_RESERVA ORDER BY id_bitacora DESC.
4. Dejar abierto también docs/auditoria/04_demo_concurrencia_auditoria.sql por si el docente pide ver el bloqueo en dos ventanas.

2.2 API REST

5. En HotelReservas.Api/appsettings.json revisar la cadena de conexión (Server=localhost;Database=HotelReservas;Trusted_Connection=True;TrustServerCertificate=True).
6. Abrir una terminal en HotelReservas.Api y ejecutar: dotnet run --urls "http://0.0.0.0:5080". El 0.0.0.0 es obligatorio para que el teléfono pueda llegar a la API; con localhost solo responde en el propio computador.
7. Abrir http://localhost:5080/swagger en el navegador del computador y probar POST /api/auth/login con recepcion1@hotel.com y Hotel2026*. Debe devolver un token.
8. Permitir el puerto 5080 en el firewall de Windows (Panel de control, Firewall, Reglas de entrada, nueva regla de puerto TCP 5080) o desactivar temporalmente el firewall en la red privada durante la auditoría.
9. Averiguar la dirección IP del computador con ipconfig (adaptador Wi-Fi, "Dirección IPv4", por ejemplo 192.168.1.10). Esa dirección se escribe en la APK.

2.3 Teléfono y APK

10. Compilar la APK v0.8 en Android Studio (Build > Build APK) e instalarla en el teléfono. La versión incluye el permiso de internet y el tráfico HTTP en red local.
11. Conectar el teléfono a la misma red Wi-Fi del computador. Si la red de la universidad aísla los dispositivos, usar el punto de acceso (hotspot) del teléfono y conectar el computador a él; en ese caso la IP del computador cambia y hay que volver a consultarla con ipconfig.

12. En la pantalla de inicio de sesión de la APK marcar "Conectar con la API", escribir http://IP:5080 y tocar "Probar conexión". Debe aparecer "API en línea".
13. Proyectar el teléfono con scrcpy o con la cámara del computador; si no es posible, el prototipo interactivo docs/auditoria/Prototipo_Interactivo.html abierto en Chrome del computador tiene el mismo modo conectado (usar http://localhost:5080).

2.4 Opcional: servicio inteligente

Si se quiere mostrar la red neuronal como servicio, en servicio_inteligente ejecutar pip install -r requirements.txt, python entrenar_modelo.py y uvicorn api:app --port 8000; la documentación queda en http://localhost:8000/docs. No es indispensable para la auditoría: la APK calcula el riesgo de cancelación en el propio teléfono.

3. Guion de la demostración funcional (numeral 2.3)

Duración prevista: doce minutos. El orden sigue los siete puntos de la guía. Cada paso indica quién lo ejecuta, qué se hace y la frase que conviene decir. Gustavo lleva el teléfono, Jonathan el computador con SSMS y la terminal de la API, Hugo explica mientras los otros ejecutan y responde el punto de integración.

Paso de la guía	Quién	Qué se hace y qué se dice
1. Inicio de la aplicación	Jonathan y Gustavo	Mostrar la terminal con la API en ejecución (dotnet run) y Swagger. Abrir la APK en el teléfono, marcar "Conectar con la API" y tocar "Probar conexión": aparece "API en línea". Decir: "La app corre en Android, la API en ASP.NET Core y los datos en SQL Server; los tres están en ejecución."
2. Navegabilidad	Gustavo	Iniciar sesión como recepcion1@hotel.com. Recorrer las pestañas Inicio, Reservas, Habitaciones, Historial, Asistente y Predicción. Decir: "El perfil decide lo que se ve: el administrador tiene además la pestaña Admin."
3. Registro de información	Gustavo	Reservas: tipo Doble, dos personas, fechas de la próxima semana, Buscar. Tocar una habitación disponible. Escribir un documento nuevo (por ejemplo el del docente), nombre, teléfono, medio de pago tarjeta, Confirmar. Aparece "HTTP 201 Created" con el código RSV y el comprobante CMP. Decir: "La app envió POST /api/huespedes y POST /api/reservas; el procedimiento sp_ConfirmarReserva insertó reserva, pago, comprobante y bitácora en una sola transacción."
4. Persistencia	Jonathan	En SSMS ejecutar las consultas preparadas: la reserva recién creada aparece en RESERVA con subtotal, impuesto y valor_total, el huésped en HUESPED, el pago en PAGO y la fila "- → CONFIRMADA" en BITACORA_RESERVA con el usuario y la hora. Decir: "Este es el dato que acaba de entrar por el teléfono; no está en memoria, está en la tabla."
5. Consulta de información	Gustavo	Historial: la lista llega de GET /api/reservas. Tocar "Ver detalle" en la reserva nueva: pagos, comprobante y bitácora vienen de GET /api/reservas/{codigo}. Volver a Reservas y buscar las mismas fechas: la habitación aparece "Reservada". Decir: "La disponibilidad la calcula la API con la consulta de solapamiento sobre RESERVA."
6. Integración	Hugo	Con el botón "Ver servidor" de la APK mostrar el registro de llamadas (método, URL, código de respuesta). Explicar el recorrido: pantalla (HTML en el WebView) → fetch con token JWT → controlador de la API → ServicioReservas → Entity Framework / procedimiento almacenado → SQL Server → respuesta JSON → pantalla. Señalar en el código ReservasController.Confirmar y ServicioReservas.ConfirmarAsync.
7. Validación	Gustavo y Jonathan	Mostrar cuatro errores controlados: contraseña incorrecta (401 en pantalla); fecha de salida anterior a la de entrada (mensaje antes de llamar a la API); documento repetido al crear huésped (409 desde la API); y la más importante: intentar reservar la misma habitación y

Paso de la guía	Quién	Qué se hace y qué se dice
		fechas que en el paso 3: la API responde 409, la app muestra que la transacción hizo ROLLBACK y en SSMS se comprueba que no entró ninguna fila nueva. Si hay tiempo, la prueba de dos ventanas en SSMS con el script 04.

Cierre (un minuto, Hugo): "Lo que mostramos conecta las tres capas del documento: la interfaz móvil, la API con la lógica de negocio y la base de datos con la transacción que evita la sobreventa. Lo pendiente está en el cronograma de la sección 18: las pantallas nativas en Android Studio desarrolladas por nosotros, el perfil de cliente sobre la API y las pruebas de los requisitos no funcionales sobre SQL Server."

3.1 Plan B si algo falla

- El teléfono no llega a la API: usar el prototipo interactivo en Chrome del computador con http://localhost:5080 en el modo conectado; la demostración es idéntica.
- La API no arranca: demostrar el registro y la consulta directamente en Swagger (POST /api/reservas, GET /api/reservas) y en SSMS; la APK se muestra en modo demostración local para la navegabilidad.
- SQL Server no responde: reiniciar el servicio (services.msc, SQL Server (MSSQLSERVER)); si no hay solución, mostrar la APK en modo local y el registro simulado del servidor, aclarando que es simulación.
- Sin red Wi-Fi: hotspot del teléfono; la IP del computador cambia (ipconfig) y se actualiza en la APK.

4. Explicación técnica (numeral 2.4): preguntas y respuestas

Las siete preguntas de la guía con una respuesta corta cada una. Conviene que cada integrante domine las tres suyas y pueda responder cualquiera de las otras.

Cómo está organizada la aplicación (Gustavo). En tres capas separadas en carpetas del repositorio: la aplicación móvil Android (HotelReservasDemo_AndroidStudio) con la interfaz; la API REST en ASP.NET Core 8 (HotelReservas.Api) con la lógica de negocio y el acceso a datos mediante Entity Framework Core; y la base de datos SQL Server creada con los scripts de la carpeta database. La app nunca toca la base de datos: todo pasa por la API con un token JWT. Además existe el servicio inteligente en Python (TensorFlow y FastAPI) al que la API reenvía las predicciones.

Qué responsabilidades tiene el backend (Jonathan). Autenticar usuarios y emitir el token con el rol; validar las solicitudes (fechas, capacidad del tipo, existencia del huésped); calcular disponibilidad y precios con IVA; ejecutar la confirmación de la reserva como una transacción con el procedimiento sp_ConfirmarReserva (verificación bajo bloqueo, inserción de reserva, pago, comprobante y bitácora, o ROLLBACK con error 50001 que se traduce a 409); cambiar estados con concurrencia optimista (ROWVERSION, error 50012); generar el comprobante compartible; y exponer el asistente de consultas y las predicciones.

Cómo se estructura la base de datos (Jonathan). Nueve tablas del modelo relacional de la sección 7: USUARIO, HUESPED, TIPO_HABITACION, HABITACION, TARIFA, RESERVA, PAGO, COMPROBANTE y BITACORA_RESERVA, más CONSULTA_ASISTENTE del script 06. RESERVA tiene restricciones CHECK de fechas y estados, columna ROWVERSION, columnas de subtotal, impuesto y valor_total con la regla valor_total = subtotal + impuesto, y la columna riesgo_cancelacion del script 07. La integridad de la reserva se garantiza con el procedimiento almacenado y el nivel de aislamiento SERIALIZABLE con UPDLOCK y HOLDLOCK.

Cómo se realiza la conexión entre los componentes (Hugo). La APK carga la interfaz en un WebView y usa fetch para llamar a la API por HTTP en la red local (la dirección se configura en la pantalla de inicio de sesión y se guarda en el teléfono). Cada llamada lleva el encabezado Authorization: Bearer con el token. La API se conecta a SQL Server con la cadena de conexión de appsettings.json a través del DbContext de Entity Framework Core; la confirmación llama al procedimiento almacenado. Las respuestas son JSON con códigos HTTP: 200, 201, 400, 401, 404 y 409.

Qué operaciones funcionan actualmente (Gustavo). Sobre la API y SQL Server: inicio de sesión, consulta de disponibilidad, consulta y registro de huéspedes, confirmación de reserva con pago y comprobante, historial con detalle y bitácora, check-in, check-out y cancelación, comprobante por WhatsApp, asistente y predicciones. En la app, además, en modo demostración: perfil de cliente con reserva pendiente y pago posterior, administración de usuarios, clientes, tarifas y habitaciones, imágenes de las habitaciones y las dos redes neuronales.

Qué dificultades se han presentado y cómo se abordaron (Hugo). Tres principales. La compilación de la app .NET MAUI en los equipos del grupo, resuelta con un proyecto Android Studio (Kotlin y WebView) que empaqueta la misma interfaz. La prueba de concurrencia, que no

se puede ver con un solo usuario, resuelta con el script de dos ventanas en SSMS y con la respuesta 409 de la API. Y la observación de la primera auditoría sobre evidencias y coherencia, atendida con la sección 16 del documento (matriz de pruebas, evidencias y tabla de coherencia) y con este modo conectado, que reemplaza la simulación por llamadas reales.

Qué funcionalidades quedan pendientes (los tres). Según el cronograma de la sección 18: las pantallas definitivas desarrolladas por el equipo en Android Studio con XML y Kotlin (compromiso de la primera auditoría, actividad 3), la administración de habitaciones y tarifas en la API (actividad 4), el perfil de cliente y la reserva pendiente sobre la API, el envío de correo desde la cuenta del hotel con la clave de aplicación, el reentrenamiento de los modelos con el historial real (actividad 7), las pruebas de aceptación de los RNF sobre SQL Server con evidencias (actividades 8 a 10) y la APK firmada para la sustentación del 27 y 28 de noviembre.

4.1 Preguntas probables del docente

¿Dónde está la transacción? En database/02_transaccion_confirmar_reserva.sql (sp_ConfirmarReserva) y la llama ServicioReservas.ConfirmarAsync; la API la expone en POST /api/reservas.

¿Qué pasa si dos recepcionistas reservan la última habitación a la vez? La primera transacción toma el bloqueo UPDLOCK/HOLDLOCK sobre las filas del rango; la segunda espera, encuentra el solapamiento y hace ROLLBACK con THROW 50001; la API responde 409 y la app pide consultar de nuevo. No queda ninguna fila a medias.

¿Cómo saben que el dato se guardó? Lo consultamos en SSMS en vivo y la app lo lee de nuevo con GET /api/reservas; el código RSV coincide en los dos lados.

¿Por qué un WebView y no pantallas nativas? Fue la forma de validar los flujos y las reglas con rapidez; la recomendación de la primera auditoría fue hacer las interfaces nosotros y quedó como actividad 3 del cronograma con reparto por integrante.

¿Cómo se protege la contraseña? Se guarda con PBKDF2-SHA256 (script 06, clase HashClave); el login compara el hash y emite un JWT con vigencia de 480 minutos.

¿Cómo se calcula el IVA? En el procedimiento almacenado: subtotal = noches por tarifa vigente, impuesto = 19 % salvo huésped extranjero no residente (artículo 481 del Estatuto Tributario), valor_total = subtotal + impuesto con una restricción CHECK.

¿Para qué sirve la red neuronal? Estima el riesgo de cancelación de cada reserva antes de confirmarla (RF19) y pronostica la ocupación (RF21); evaluadas con exactitud, precisión, sensibilidad, F1, MAE, MSE, RMSE y R^2 (sección 15.5).

¿Qué pruebas tienen? Diez pruebas automáticas del servicio inteligente (pruebas/evidencias.json), pruebas con Playwright del prototipo y la matriz de RNF de la Tabla 11 con su estado real: lo que falta ejecutar sobre SQL Server está marcado como parcial.

5. Lista de verificación para el día

- Computador con SQL Server en ejecución y la base HotelReservas con datos de prueba (scripts 01 a 07).
- API ejecutándose con dotnet run --urls "http://0.0.0.0:5080"; Swagger abierto; puerto permitido en el firewall.
- IP del computador anotada; teléfono en la misma red; APK v0.8 instalada con "Conectar con la API" marcado y "Probar conexión" en verde.
- SSMS con la ventana de consultas preparada (RESERVA, HUESPED, PAGO, BITACORA_RESERVA) y el script 04 abierto.
- Documento maestro en PDF (110 páginas) abierto en el computador y copia en una memoria USB.
- Repositorio abierto en el navegador: https://github.com/gustavo8015/HotelReservas, con el README y el historial de commits a la vista.
- Cada integrante con sus tres preguntas de la sección 4 preparadas y el código de su parte abierto en el editor (Gustavo: prototipo.html y MainActivity.kt; Jonathan: ReservasController.cs, ServicioReservas.cs y scripts SQL; Hugo: Program.cs, HotelReservasContext.cs y NotificacionesController.cs).
- Cargador del teléfono y cable; scrcpy instalado si se va a proyectar el teléfono.

Nota de honestidad para la auditoría: decir con claridad qué corre sobre la base de datos (todo lo que se demuestra en el modo conectado) y qué sigue siendo simulación en la app (el perfil de cliente, la administración de usuarios y tarifas, el pago posterior). La primera auditoría bajó la nota precisamente por no diferenciar lo implementado de lo pendiente; esta vez la diferencia se muestra con la casilla "Conectar con la API" y con la consulta en SSMS.